# RoboDex: Vision-Language Driven Robotic Simulator for Dexterous Manipulation Training

Yunhai Han[1], Kin Man Lee[1], Manisha Natarajan[1], Yixin Zhang[1]
[1]Georgia Institute of Technology
yhan389, klee863, mnatarajan30, yzhang4179@gatech.edu

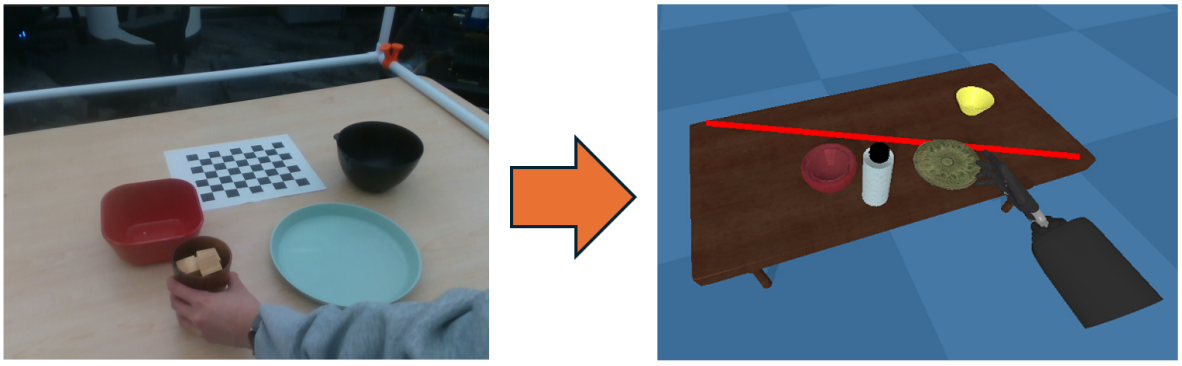


Figure 1: We propose a novel framework called **RoboDex** to train dexterous manipulation policies from human videos. This figure depicts an example scene reconstructed in simulation.

## Abstract

*With an aging population and a growing shortage of caregivers, the need for in-home robots is increasing. However, it is still intractable for robots to have all functionalities built-in before deployment and directly adapt to diverse household settings and user preferences. To address this challenge, we propose a novel robot learning framework called **RoboDex** that facilitates learning dexterous manipulation policies from a single human demonstration video per task. Leveraging pre-trained vision-language models (VLMs), RoboDex generates task descriptions and 3D scene graphs to recreate the task setup in simulation. Upon constructing the simulation scene, RoboDex uses pre-trained large language models (LLMs), such as GPT, to decompose the task and generate reward functions, facilitating automated training in simulation. Through individual experiments, we demonstrate the effectiveness of each core component of RoboDex and showcase how the trained policy successfully performs the demonstrated task within a simulation environment that closely mimics real-world scenarios. Future work involves incorporating the sim-to-real transfer of learning techniques to validate RoboDex's efficacy in real-world deployments.*

## 1. Introduction

The aging population and shortage of caregivers [1] raise an urgent demand for robots to assist the elderly with their daily activities at home. However, unlike the widespread utility of large language models (LLMs) in tasks like translation and text refinement, robot learning has yet to achieve similar deployment, with robots still predominantly operating in highly structured lab or industrial environments. This is primarily due to (i) the limited scale of training data—household robots must handle diverse tasks and environments, making comprehensive data collection highly challenging; (ii) the difficulty of ensuring safety in out-of-distribution tasks during inference poses significant risks for real-world manipulation despite training on large-scale datasets; and (iii) users may have unique preferences for how the robot should assist in the home, which can be conveyed through demonstrations. Thus, we propose to leverage human demonstration videos (collected from the internet or user-specific demonstrations, i.e., *low-privileged* demonstration data without action labels) to train a *specialized* dexterous[1] robot agent for different manipulation tasks

[1]A parallel gripper could potentially achieve this goal through a different approach. However, in this work, we assume the need for a dexterous agent, supported by the following: (i) dexterous hands are better suited for tool use, and (ii) multi-finger coordination helps finer-grained control.

using an *automated* learning process. Our approach, **RoboDex**, adopts a *Real-to-Sim-to-Real* approach, where we first create a simulation environment from demonstration videos (see Figure. 1) and use motion re-targeting with reinforcement learning to learn dexterous policies and transfer to real-world using prior work [2].

Dexterous manipulation for robots is a highly challenging problem, and several approaches have been proposed that focus on (i) adopting simple model architectures and training pipelines for diverse manipulation tasks [3], (ii) efficiently acquiring manipulation skills from low-privileged demonstration data using a universal reward strategy in RL [2], (iii) identifying visual features from the same demonstration data [4], and (iv) ensuring the operational safety of the learned skills during inference [5]. While these works appear to lay the foundation for the proposed approach, it is important to note that the absence of essential contact information in low-privileged demonstration data (i.e., human videos) necessitates the use of an expert-designed robot simulator [6] for each specific task in the policy learning process [2]. Building on this, we aim to automatically generate the robot simulator from the same demonstration data by leveraging the reasoning and generative capabilities of VLMs and LLMs.

Along these lines, several works have also addressed simulator generation for policy learning [7, 8, 9, 10, 11, 12]. However, in [7, 9], the focus is on generating simulation data from language descriptions using heuristic-based planners, followed by robot policy imitation learning on the synthetic dataset. On the other hand, works in [8, 10, 11] aim to generate robot simulators that support RL learning, enabling the learning of specific robot skills for a given task. However, in these works, the reward strategies are directly generated from language models, which are simplistic and likely insufficient for training tasks involving dexterous hands, due to the complexity of the contact patterns and the high-dimensional observation and action spaces. Additionally, the work in [12] is similar to our approach, as they generate a simulator from human manipulation videos and apply reinforcement learning specifically for dexterous hands. However, they assume access to an offline library of manipulation objects and the knowledge of which objects are being manipulated, which limits its applicability in *open-world* manipulation scenarios, where neither offline object assets nor pre-defined object labels are available.

In this work, we first leverage an off-the-shelf language model, such as GPT-3.5 used in [10], to generate simulation code from language-based task descriptions. To create a specific simulator that enables the robot to practice the demonstrated skills from the human in a "digital twin" simulation environment, we further integrate two additional off-the-shelf VLMs [13, 14]. These models can reason about the task's intent from human demonstrations [13] and analyze the objects and their spatial configurations in the scene [14], respectively. More specifically, we downsample human manipulation videos into several frames and query a VLM to describe the human's actions and the objects being manipulated or present in the scene. Next, we use the RGBD streams from the same manipulation video to reconstruct the real-world scene, in which we leverage a CLIP-based open-vocabulary method [15] and a point cloud fusion method to identify, segment, and locate each object while dynamically generating their assets. After generating the simulator, we can train the dexterous robot agent via RL using either: (i) the reward function generated by the LLM model [10], or (ii) the retargeted human and object trajectories as the reward signal [2] (future works). Note that for both, they do not need task-specific reward shaping from the experts. Finally, we employ domain randomization techniques [16] to enable zero-shot transfer of the trained RL policy from simulation to the real-world setting, allowing the robot to emulate the human demonstration observed in the video (one-shot imitation from video).

In summary, **RoboDex** presents a *Real-to-Sim-to-Real* robot learning framework that enables: i) automatic generation of a "digital twin" simulator from human manipulation videos, ii) efficient skill acquisition through the generated simulator using a universal reward strategy, and iii) zero-shot transfer of the learned skills to real-world settings. We want to emphasize that the entire learning process does not require any expert involvement and only requires *a single demonstration video per task* (unlike most prior work on learning from demonstration approaches [17]), making it more usable for non-experts. To the best of our knowledge, no existing work can achieve the same objectives. We believe that our work can pave the way for democratizing robotics—allowing non-experts themselves to teach robots for their specific needs, and can potentially lead to the development of an autonomous open-world robot agent.

## 2. Related Work

In this section, we contextualize our contributions within three relevant sub-fields: (i) learning from human demonstrations for dexterous tasks, (ii) simulation and scene generation for robotics, and (iii) leveraging foundation models for robot learning.

### 2.1. Learning Dexterous Skills from Human Demonstrations

Given the abundance of video data on the internet and the advances in motion retargetting (e.g., [18] and [19]), recent research has focused on learning dexterous manipulation skills from human videos. Among them, a key challenge is the lack of the action labels that speaks to the contact effects between the robot, the object, and the environment. Existing works tackle this challenge in one of

two ways. Some methods leverage additional in-domain teleoperated demonstrations (or post-hoc corrections) from an expert [19, 20, 21, 22, 23, 24]. Other methods provide robots an opportunity to learn from in-domain interactions, either in simulation [2, 18, 25, 26, 27] or in the real world [28, 29, 30]. However, developing an *open-world, in-the-wild* robot learning system from human manipulation videos poses significant additional challenges. The first solution is unlikely to be viable due to the absence of teleoperation systems in most homes and users' lack of expertise in robot systems. Similarly, enabling robots to interact directly with the real world is often impractical, as it typically requires RL fine-tuning, which either poses potential risks in physical environments or demands continuous human supervision. As a result, providing robot interactions in simulation and then transferring the learned skills to the real world emerges as the most promising approach. However, existing works largely assume access to pre-built, task-specific simulators, which limits their applicability to the challenges considered in this work. To overcome this, we propose to construct the simulation scene directly from the demonstration video in an automated manner.

### 2.2. Simulation & Scene Generation

Generating simulator or simulation data for robot policy learning has recently been an attended topic [7, 8, 9, 10, 11, 12]. Among them, in [7, 9], the focus is on generating simulation data and then utilize the imitation learning methods on the synthetic dataset to distill the robot policy. On the other hand, works in [8, 10, 11] aim to generate robot simulators that support subsequent RL learning, enabling the learning of specific robot skills for a given task. However, in these works, the reward functions are directly generated from language models, which are likely insufficient for dexterous manipulation skills [6]. Though [12] seems applicable in our application, they assume access to an offline library of object assets and the knowledge of which objects are being manipulated, limiting its extension to the open-world setting. Also, they do not validate their works on hardware[2].

In the context of scene generation for creating a 'digital twin'-like simulator, several existing works have demonstrated the effectiveness of dynamically generating objects from real-world data [11, 14, 31, 32, 33, 34]. In contrast to them, this work focuses on designing a simulator specifically tailored for dexterous manipulation learning using the aforementioned method.

### 2.3. Foundation Models in Robotics

Recent efforts have leveraged foundation models in robot learning. For instance, [35] integrates the semantic understanding of the CLIP model [15] with the spatial reasoning of Transporter [36] to train a language-conditioned robot imitation policy using expert demonstrations, which can further effectively generate manipulation plans (*middle-level*). More recently, Google's work on distilling a large-scale robot foundation model from diverse robotic datasets [37] has shown its efficacy in directly generating robot actions (*low-level*), demonstrating emergent capabilities for generalization to new domains and tasks. Similarly, [38] adopts a comparable approach, leveraging a smaller dataset but utilizing a more structured policy architecture. In addition to these efforts, researchers have also explored leveraging language foundation models to generate robot simulators (*high-level*) [7, 8, 9, 10], which can be used for subsequent robot policy learning. Our work falls under this category but adopts a different approach to policy learning.

## 3. Methods

In this section, we provide an overview of our approach, **RoboDex**, to train a robot agent for dexterous manipulation tasks from human demonstration videos. Our framework involves reconstructing the task scene in simulation using natural language task description extracted using pre-trained VLMs (Section 3.1) and 3D scene description (Section 3.2) extracted from the demonstration videos. Upon extracting the relevant objects and their descriptions from the video, we then leverage GPT-3.5 to reconstruct the scene in simulation (Section 3.4). Currently, we rely on open-source 3D object assets for simulation scene reconstruction. We also experimented with 3D asset generation corresponding to each object in the scene (Section 3.3) and aim to incorporate these generated assets directly into the simulation scene in future work. After reconstructing the scene in simulation, we use reinforcement learning (RL) along with movement primitives for training robot policies (Section 3.5), similar to prior work [10]. Please note that our current work has only been tested on tabletop manipulation settings. An overview of our approach is provided in Figure. 2.

### 3.1. Natural Language Task Description

To provide the most task-relevant language descriptions for querying the LLMs for simulator generation, we first utilized a pre-trained VLM model [13] with 36 uniformly downsampled frames from the human demonstration videos. For VLM prompts, we developed three options:

- *little prompt*: "Summarize key manipulative actions and interactions from a demonstration video"
- *rough prompt*: "Summarize hand-object interactions in the video. " "Detail any object-object interactions observed in the demonstration. " "Identify and describe any distractors in the video."

[2]We plan to validate our method on hardware in the near future and potentially submit a full conference paper.

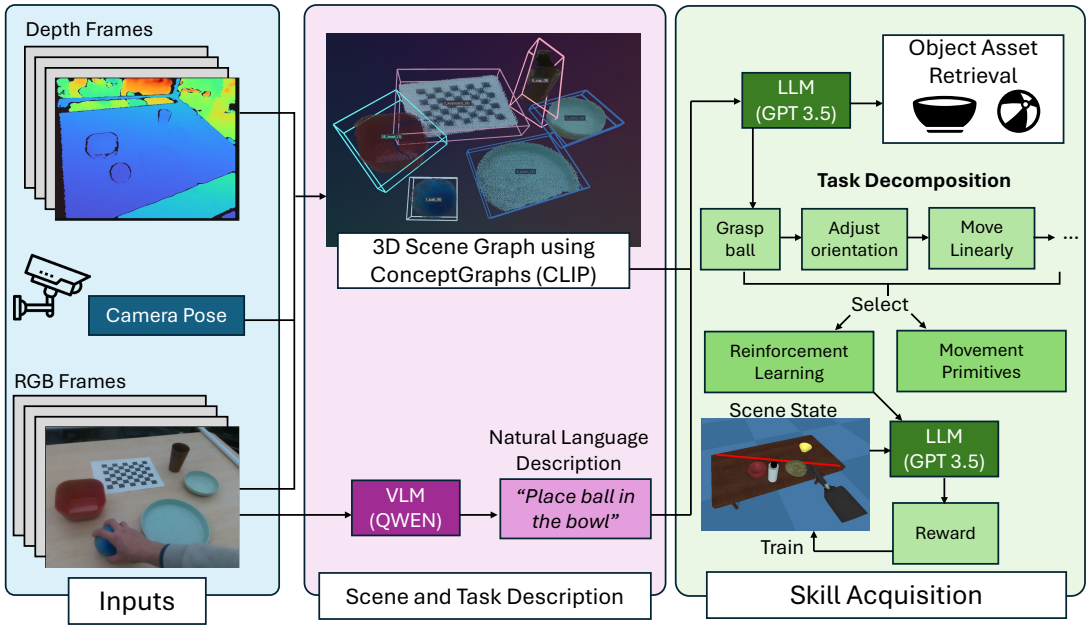



Figure 2: An overview of the proposed **RoboDex** framework. **RoboDex** is a novel approach to train dexterous manipulation policies directly from human videos. We first reconstruct the task scene in simulation using natural language description (from pre-trained QWEN) and 3D positions and object descriptions. The 3D pose and object descriptions are identified using CLIP [15] from RGB-D sequence, with a prior framework [14]. The extracted object descriptions and the task description from VLM are then fed into a pre-trained LLM (GPT-3.5) to retrieve object assets from Objaverse and reconstruct in PyBullet simulator. The LLM also decomposes the task and generates reward functions for training different subtasks.

- *detailed prompt*: "Analyze the provided human demonstration video and generate a detailed task description focusing on the manipulation process. Specifically:" "1. Identify and describe instances of hand-object interaction, where the human interacts directly with objects using their hands (e.g., grasping, moving, or manipulating an object)." "2. Identify and describe instances of object-object interaction, where the human uses one object to manipulate or affect another (e.g., using a tool to perform a task, )." "3. Identify and describe instances of object-object relationship that has influence to the tasks, such as cubes in a cup )." "4. Pay special attention to distractors in the scene—objects, movements, or actions that are unrelated to the main task but may appear in the video. These should be distinguished from relevant actions." "Provide a structured description of the task, highlighting the sequence of key actions, the relationships between objects, and any contextual factors affecting the manipulation process."

An example output from this component could be: "Place the ball in the bowl. Move the bowl to the right side of the table." (using *little prompt* option). This output will then be used to query the LLM for further simulation generation (Section. 3.4). Through experiments, we found that *little prompt* already be good enough for our purpose. Therefore, the results reported in the Section. 4.2.3 are obtained using the *little prompt* option.

### 3.2. Scene Description

Scene representation is one of the key design choices that can enable robots to perform a wide variety of tasks under various domains. Specifically, in our approach, we aim to create a scene representation that is immediately applicable to reconstruct scenes for existing robot simulators, such as PyBullet [39] or IsaacSim [40]. As our work aims to enable non-expert end-users to teach robots, our scene representation should not be limited to only making inferences from a set of pre-defined objects but should also be capable of handling new objects, i.e., open-vocabulary representations and must be scalable to various number of object entities in the environment (since household environments can have varying levels of clutter). Therefore, we leverage a recent prior work, called ConceptGraphs, that aims to build an open-vocabulary graph representation for 3D scenes [14].

ConceptGraphs incrementally constructs a 3D map and a 3D scene graph, $\mathcal{M}_t =< \mathbf{O}_t, \mathbf{E}_t >$ from a set of RGB-D frames with camera pose information. Here, $\mathbf{O}_t = \{o_j\}_{j=1...J}$ represents the set of objects and $E_t =$

$\{e_k\}_{k=1...K}$ represents the set of edges. Each node, $o_j$, in the scene graph refers to an object in the scene and is represented as a 3D point cloud, $p_{o_j}$ with semantic features, $p_{o_j}$. To extract the objects in the scene, the approach involves first running a class-agnostic segmentation using the Segment Anything Model (SAM) [41] to obtain a set of masks, $\{\mathbf{m}_{t,i}\}i = 1...M$. Each mask is passed to the CLIP model [15] along with the RGB image frame, $I_t$, to obtain a feature vector, $\mathbf{f}_{t,i} = \text{Embed}(I_t, \mathbf{m}_{t,i})$. Each masked region is then projected to 3D, denoised using DBSCAN clustering, and transformed to the map frame, resulting in point cloud, $p_{t,i}$ and feature vector, $\mathbf{f}_{t,i}$. As the map is built incrementally, any newly detected object is compared with a similarity metric based on distance and feature similarity to determine whether it is a new object or associated with any of the mapped objects. Lastly, once the entire image sequence is processed, the CLIP model [15] is used to generate captions for the nodes in the graph (based on the extracted feature vectors) by passing in cropped images of the object and determine the semantic relationships (or the edges) of the graph through object captions and positions.

In our approach, we first extract the camera pose using checkerboard-based calibration. The extracted pose information is then fed as input along with the RGB-D camera frames to construct the 3D scene graph using ConceptGraphs. Finally, the object positions, captions, and the spatial relationships between objects are extracted from the 3D scene graph for reconstructing the scene in simulation.

### 3.3. On-the-fly Object Asset Generation

As previously noted, predefining manipulation objects is infeasible for a robot agent operating in an open-world environment, making it impractical to rely on an offline object asset library. To address this, our approach employs a method [42] to dynamically generate object assets from the demonstration images, allowing them to be directly imported into the simulator.

First, the input image is preprocessed to extract depth and normal information, providing essential guidance for generating the 3D model. Leveraging a pre-trained instant-ngp NeRF backbone [43], multiple viewpoints are synthesized to form an initial 3D representation that closely aligns with the input object's geometry. Next, diffusion models: Stable Diffusion [44] and Zero-1-to-3 [45] are employed to refine and enhance the model's appearance, ensuring fidelity to the source image and adherence to its visual cues. Finally, DMTet [46]is utilized during inference to increase resolution and detail, resulting in a geometrically consistent reconstruction with smooth, realistic textures across all viewpoints.

### 3.4. Simulator Generation

Once we have extracted the information about the objects and their spatial relationships, we can reconstruct the scene in simulation. We use an LLM (GPT-3.5) to generate the YAML configuration files that contain the attributes of the relevant assets based on the natural language task description. We build on top of the asset selection and scene configuration components of the scene generation framework that was proposed in RoboGen [10]. We describe the modifications made to each component in detail below.

**Relevant Assets** RoboGen [10] was focused on developing a generative robotic agent that can learn different tasks by creating complex and diverse scenes. Therefore, their approach involved generating different simulation scenes using LLMs, that are plausible in the real world by introducing distractor objects irrelevant to the task to train more robust policies. In contrast, our approach aims to reconstruct the scene from the human demonstration video accurately. Therefore, we extract relevant object information from the scene (as described in Section 3.2) and then query open-source, large-scale 3D asset libraries to retrieve the objects that are most similar to the object descriptions provided from both the task and scene descriptions. We rely on two large-scale datasets for simulation scene generation, namely the Objaverse [47] and PartNet-Mobility [48] library. For objects that are manipulated in the video demonstration, we need articulated objects (with joint links, etc.) that are recovered from the PartNet Mobility dataset. For background objects, we also leverage the Objaverse dataset, which includes a much larger collection of 3D assets.

**Scene Configuration** For RoboGen, the spatial configuration of all objects are determined by the LLM for a plausible environment. In the case of scene reconstruction for our tasks, the spatial configuration and size of the relevant assets are determined based on the exact Cartesian position coordinates (i.e., $x$, $y$) of the centroid and bounding box provided from the scene descriptor (Section 3.2). Since we limit our tasks to the tabletop setting, all objects are initialized to be on top of the table (i.e., initial object height $z$ is constrained to the height of the table).

### 3.5. Skill Acquisition

Similar to RoboGen, we query the LLM to decompose the task into a series of subtasks and the method to solve each subtask, given a natural language task description. Each subtask can be solved through a motion planning action primitive [49] for rough, general motions involving collision-avoidance or reinforcement learning for contact-rich manipulation.


Figure 3: We consider two manipulation tasks: **Task 1**: Picking up a blue ball and placing it onto the first plate on the right; **Task 2**: Picking up a bottle filled with cubes and pouring the contents into a black bowl.

The action primitive is used for the robot to move into a favorable position to manipulate the target object. A position near the target object is sampled as the end-effector goal, which is used as input to the planner. The trajectory from the planner is executed if a collision-free solution is found, otherwise this process is repeated by resampling another position.

For the subtasks involving reinforcement learning, the LLM is prompted to generate the corresponding reward functions. The LLM is provided with a list of helper functions from the simulator API to acquire state information of the scene (e.g. object positions/orientations, end-effector positions/orientations) and in-context examples as a prefix prompt. Note that instead of relying on generated reward functions, an alternative strategy is to leverage the retargeted robot hand and object movements to design tracking-based rewards, as detailed in [2]. We plan to incorporate this approach in future work.

## 4. Experiments, Results, Ablations

We start by outlining the experimental setup and then demonstrate the efficacy of our approach.

### 4.1. Experiment Setup

**Hardware Setup:** Our approach involves learning from human demonstration videos. Currently, we focus on tabletop settings for learning dexterous manipulation tasks. Our hardware setup involves using the Intel RealSense camera mounted above the table to collect demonstration videos. Currently, we collected two human demonstration videos for pick-place tasks with various household objects with a static camera, as each shown in Figure. 3. Note that each video is used to train a separate robot agent. Additionally, we collected four non-manipulation videos to further evaluate the quality of scene generation and the dynamic object asset creation process.

| Scene | # Objects Identified | Object Labels (From CLIP) | Object Labels (Human) | Pose Error (in cm) |
|---|---|---|---|---|
| 1 | 4 | bowl, bowl, soap dish, cup | bowl, plate, bowl, cup | $4.67 \pm 7.33$ |
| 2 | 3 | ---, ball, cup, ball | bowl, ball, cup, ball | $8.81 \pm 7.05$ |
| 3 | 3 | ---, bowl, bowl, cup | bowl, plate, bowl, cup | $1.47 \pm 0.26$ |
| 4 | 4 | cup, ball, ball, bowl | cup, ball, ball, bowl | $2.36 \pm 0.79$ |

Table 1: Pose Estimation error from 3D scene reconstruction. We examined pose estimation of different household objects placed at pre-defined markers for four scenes. Each scene included four objects at different locations. The blanks in object labels indicate failure to identify object.

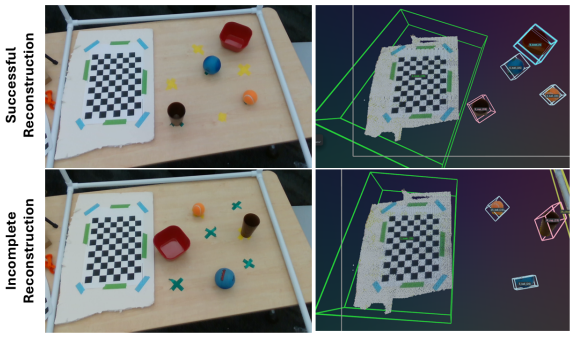



Figure 4: We provide an example of successful and incomplete 3D scene reconstruction, which is used to extract object pose information. In the above image, all four objects are correctly identified, whereas in the image below, only three of the four objects are identified (missing red cup), and the fourth object only has partial point cloud data, which leads to higher inaccuracies in pose estimation.

**Training Setup:** PyBullet [39] is used as our simulation platform. We incorporated the OpenAI Gym environment of the Shadow Dexterous Hand [50] into our framework to evaluate dexterous manipulation in addition to the already supported manipulator platforms from RoboGen [10]. See Fig. 1 for an example of a reconstruct scene in the PyBullet simulation. For action primitives, the Batch Informed Trees (BIT*) [51] implementation from the Open Motion Planning Library (OMPL) [52] is used as the motion planning algorithm. Reinforcement learning policies are trained with soft-actor critic (SAC) [53].

### 4.2. Results

#### 4.2.1 Scene Reconstruction

The first step in RoboDex involves identifying different objects in the scene and extracting their spatial relationships for simulation scene generation (as described in Sec-

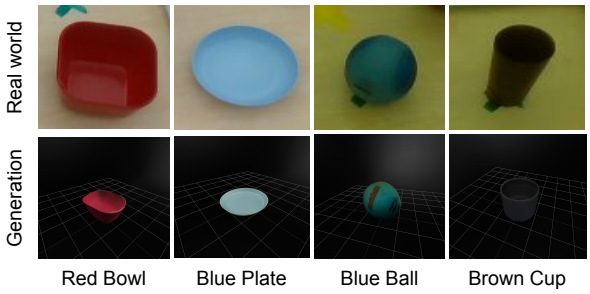



Figure 5: We select four objects used in the experiments and display both their actual appearance and the corresponding generated asset that is rendered in Blender.

tion 3.2). We use standard computer vision techniques for calibration and point cloud clustering along with a prior framework called ConceptGraphs [14] to extract relevant object information from the scene. To estimate the error in identifying the positions of different objects in the scene, we first created four static scenes with objects located at pre-defined markers (at a known distance from a fixed origin on the table). We set the origin as the bottom left of the checkerboard (used for calibration). We tested with four objects per scene and report the error in the estimated locations of different objects in Table 1.

We estimated the intrinsic and extrinsic camera parameters using calibration. However, when reconstructing the 3D map and scene graphs using [14], we needed an additional parameter for converting png to depth scale, which was unavailable from camera specifications. Therefore, we estimated this value through trial-and-error based on 3D map reconstruction and had to subsequently choose a scaling factor of $k = 2.2$ to ensure that the origin on the table was close to the origin in the 3D reconstructed scene. The distance error from the ground truth locations of the different objects was calculated after including the fixed scaling factor for all estimated poses. We also had to tune parameters for point cloud clustering in the DBSCAN algorithm.

We note that the reconstructed 3D Scene graph failed to identify one object in two of the scenes since the object (red bowl) looked like a square and had no side views. Such errors in 3D reconstruction could be mitigated by incorporating multiple views of the scene, instead of using a fixed camera. Further, one of the objects in two scenes only had partial point cloud data as it was slightly out of the field-of-view of the static camera. This also led to a higher error in pose estimation. We show a few examples of successful and failure cases for scene reconstruction in Figure. 4.

#### 4.2.2 On-the-fly Object Asset Generation

The second step in RoboDex involves dynamically generating the objects seen in the demonstration video. The input image crops, obtained during scene reconstruction, are processed as outlined in Section 3.3 to reconstruct each object asset (i.e., *.obj* object files). We selected four objects used in our experiments and present the qualitative results in Figure. 5. We observe that the generated objects closely match the actual objects. However, the generated object assets are in unit size, which means they may not accurately reflect the actual physical size of the real-world objects. As a result, we have not yet successfully imported them into the simulator. However, we believe that by utilizing the 3D bounding box size of each object, we can estimate its physical dimensions and scale the objects appropriately when loading them into the simulator (future works).

For each scene, we also display all object assets together, and specify their $x, y$ positions based on the spatial configurations detected during scene reconstruction in Figure. 6. It can be observed that, for each scene, the simulation closely mirrors the real-world setup, providing an environment where robots can effectively practice the skills required for real-world tasks.

#### 4.2.3 Simulation Generation & Policy Learning

While we have successfully generated object assets dynamically, the current simulation generation relies on pre-built offline object assets due to the normalized physical size of the generated objects and time constraints. As such, the objects in the PyBullet simulation is of the same class as detected in the scene description, but are physically different. Additionally, manipulable objects are restricted to objects in the PartNet-Mobility [48] library, so unavailable objects were remapped to existing objects in the dataset for the LLM to built the simulated scene. See Figure. 7 for qualitative differences in the simulated assets compared to the task objects in Figure. 3. Additionally, we encountered issues when attempting to implement the action primitives to the dexterous hand and evaluated the skill learning on the generated simulation on the Franka instead.

Figure. 7 shows the rollout of the skill with mixed action primitives and reinforcement learning. While the robot is able to position itself near the target object with action primitives (frames 1-3), the reinforcement learning policy fails to properly grasp and move the object to the goal location (frames 4-5). Figure. 8 shows the reward curves for decomposed reinforcement learning subtasks in both of the tasks. In Task 1, we observe that the robot is unable to learn a proper policy for Subtask 1 (adjust to orientation of the ball) and the reward for Subtask 2 (Move ball linearly to position of plate) is unstable. In Task 2, the reward plateaus for Subtask 1 (Pour the contents of the cup into the bowl), but does not successfully complete the subtask. The generated reward function for Subtask 2 encountered an error and failed to train. Overall, reinforcement learning for subtasks

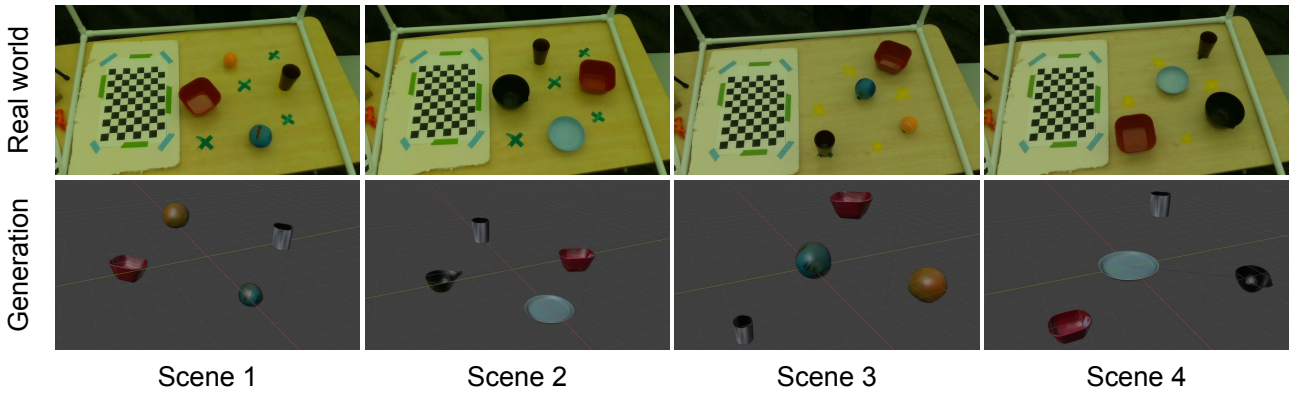
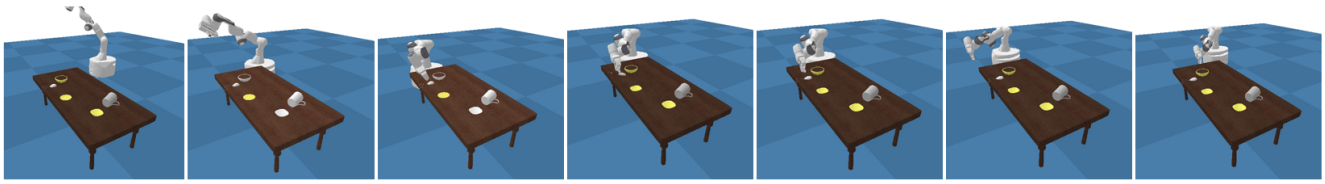
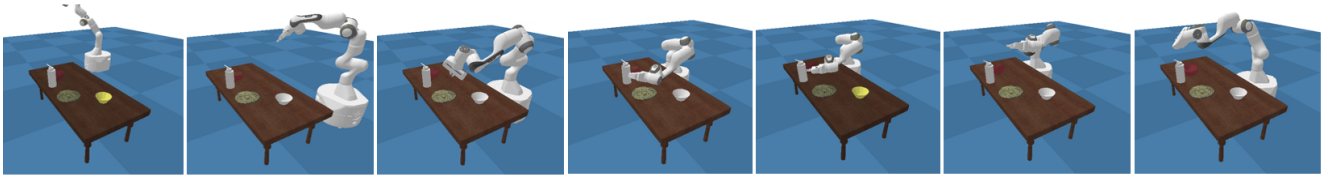



Figure 6: For each scene, we compare the real-world setting with its simulation counterpart, generated using the dynamically created object assets and extracted spatial configurations. Specifically, Blender is used to render each object asset, and its console is utilized to precisely position the objects within the simulated environment. Note that rotations are manually tuned.

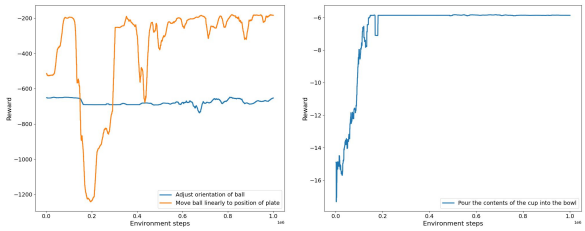


(a) Task 1 Rollout



(b) Task 2 Rollout

Figure 7: Rollout of robot policy with mixed action primitives and reinforcement learning




Task 1 RL Reward curve  Task 2 RL Reward curve

Figure 8: Rewards of the reinforcement learning subtasks in each task.

do not seem to very effective in both scenerios, highlighting the need to develop alternate reward strategies for contact-rich manipulation in generative simulations.

## 5. Discussion/Conclusion

In this work, we develop a novel robot learning framework, **RoboDex**, that can i) automatically generate a "digit twin" simulator from human manipulation videos, ii) efficiently learn the robot skills in an expert-free manner using the generated simulator, and iii) zero-shot transfer the learned skills to the real-world setting. To achieve this, we leverage off-the-shelf VLMs to generate task descriptions and scene & object descriptions from the same human demonstration video. We then input this information into another off-the-shelf LLM to reconstruct the simulation environment and generate reward functions for training the robot agent using reinforcement learning.

In future work, we aim to: i) attach a dexterous robot hand to the robot arm in the simulator, ii) import dynamically generated object assets into the simulator with the correct size, iii) utilize a tracking-based reward strategy to learn robotic dexterity skills, and iv) apply domain randomization

techniques in RL for sim-to-real transfer.

In addition to the unfinished components mentioned above, one underlying assumption is that all manipulation objects are not articulated. For articulated objects, it is more challenging to describe the kinematic properties of each part when generated on the fly. To address this, additional expert knowledge on the articulated object is necessary.

## 6. Work Division

See Table. 2.

## References


[1] U.S. Census Bureau. National Population Projections Tables: Main Series., 2023. https://www.census.gov/data/tables/2023/demo/popproj/2023-summary-tables.html. 1

[2] Yunhai Han, Zhenyang Chen, Kyle A Williams, and Harish Ravichandar. Learning prehensile dexterity by imitating and emulating state-only observations. *IEEE Robotics and Automation Letters*, 9(10):8266–8273, 2024. 2, 3, 6

[3] Yunhai Han, Mandy Xie, Ye Zhao, and Harish Ravichandar. On the utility of koopman operator theory in learning dexterous manipulation skills. In *Conference on Robot Learning*, pages 106–126. PMLR, 2023. 2

[4] Hongyi Chen, ABULIKEMU ABUDUWEILI, Aviral Agrawal, Yunhai Han, Harish Ravichandar, Changliu Liu, and Jeffrey Ichnowski. Korol: Learning visualizable object feature with koopman operator rollout for manipulation. In *8th Annual Conference on Robot Learning*. 2

[5] Hanyao Guo, Yunhai Han, and Harish Ravichandar. On the surprising effectiveness of spectrum clipping in learning stable linear dynamics. *arXiv preprint arXiv:2412.01168*, 2024. 2

[6] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations. In *Proceedings of Robotics: Science and Systems (RSS)*, 2018. 2, 3

[7] Lirui Wang, Yiyang Ling, Zhecheng Yuan, Mohit Shridhar, Chen Bao, Yuzhe Qin, Bailin Wang, Huazhe Xu, and Xiaolong Wang. Gensim: Generating robotic simulation tasks via large language models. *arXiv preprint arXiv:2310.01361*, 2023. 2, 3

[8] Pushkal Katara, Zhou Xian, and Katerina Fragkiadaki. Gen2sim: Scaling up robot learning in simulation with generative models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6672–6679. IEEE, 2024. 2, 3

[9] Pu Hua, Minghuan Liu, Annabella Macaluso, Yunfeng Lin, Weinan Zhang, Huazhe Xu, and Lirui Wang. Gensim2: Scaling robot data generation with multi-modal and reasoning llms. *arXiv preprint arXiv:2410.03645*, 2024. 2, 3

[10] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation. *arXiv preprint arXiv:2311.01455*, 2023. 2, 3, 5, 6

[11] Marcel Torne, Anthony Simeonov, Zechu Li, April Chan, Tao Chen, Abhishek Gupta, and Pulkit Agrawal. Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation. *arXiv preprint arXiv:2403.03949*, 2024. 2, 3

| Student Name | Contributed Aspects | Details |
|---|---|---|
| Yunhai Han | Literature review and Data collection | Conducted literature review on learning for dexterous manipulation; collected the human demonstration videos, and assisted in evaluating the experimental results |
| Kin Man Lee | Implementation and Analysis | Implemented the RoboGen framework on code generation and analyzed its performance on generating the configuration files for simulation scene reconstruction and reward functions |
| Manisha Natarajan | Implementation and Analysis | Implemented the scene reconstruction framework; evaluated the reconstruction accuracy and distilled the spatial configurations between objects |
| Yixin Zhang | Implementation and Analysis | Implemented the natural language task description and the on-the-fly object asset generation and evaluated the effects of using different prompts on task description and different methods on asset generation |

Table 2: Contributions of team members.


[12] Bohan Zhou, Haoqi Yuan, Yuhui Fu, and Zongqing Lu. Learning diverse bimanual dexterous manipulation skills from human demonstrations. *arXiv preprint arXiv:2410.02477*, 2024. 2, 3

[13] Qwen Team. Qwen2.5: A party of foundation models, September 2024. 2, 3

[14] Qiao Gu, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, et al. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5021–5028. IEEE, 2024. 2, 3, 4, 7

[15] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. 2, 3, 4, 5

[16] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017. 2

[17] Sumedh Sontakke, Jesse Zhang, Séb Arnold, Karl Pertsch, Erdem Bıyık, Dorsa Sadigh, Chelsea Finn, and Laurent Itti. Roboclip: One demonstration is enough to learn robot policies. *Advances in Neural Information Processing Systems*, 36, 2024. 2

[18] Yuzhe Qin, Yueh-Hua Wu, Shaowei Liu, Hanwen Jiang, Ruihan Yang, Yang Fu, and Xiaolong Wang. Dexmv: Imitation learning for dexterous manipulation from human videos. In *European Conference on Computer Vision*, 2022. 2, 3

[19] Kenneth Shaw, Shikhar Bahl, and Deepak Pathak. Videodex: Learning dexterity from internet videos. In *Conference on Robot Learning*, pages 654–665. PMLR, 2023. 2, 3

[20] Siddhant Haldar, Jyothish Pari, Anant Rai, and Lerrel Pinto. Teach a robot to fish: Versatile imitation from one minute of demonstrations. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023. 3

[21] Sridhar Pandian Arunachalam, Sneha Silwal, Ben Evans, and Lerrel Pinto. Dexterous imitation made easy: A learning-based framework for efficient dexterous manipulation. In *IEEE International conference on robotics and automation (ICRA)*, 2023. 3

[22] Yuzhe Qin, Hao Su, and Xiaolong Wang. From one hand to multiple hands: Imitation learning for dexterous manipulation from single-camera teleoperation. *IEEE Robotics and Automation Letters*, 7(4). 3

[23] Sridhar Pandian Arunachalam, Irmak Güzey, Soumith Chintala, and Lerrel Pinto. Holo-dex: Teaching dexterity with immersive mixed reality. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. 3

[24] Chen Wang, Haochen Shi, Weizhuo Wang, Ruohan Zhang, Li Fei-Fei, and C. Karen Liu. Dexcap: Scalable and portable mocap data collection system for dexterous manipulation. *arXiv preprint arXiv:2403.07788*, 2024. 3

[25] Priyanka Mandikal and Kristen Grauman. Learning dexterous grasping with object-centric visual affordances. In *IEEE international conference on robotics and automation (ICRA)*, 2021. 3

[26] Sudeep Dasari, Abhinav Gupta, and Vikash Kumar. Learning dexterous manipulation from exemplar object trajectories and pre-grasps. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. 3

[27] Ananye Agarwal, Shagun Uppal, Kenneth Shaw, and Deepak Pathak. Dexterous functional grasping. In *7th Annual Conference on Robot Learning*, 2023. 3

[28] Kelvin Xu, Zheyuan Hu, Ria Doshi, Aaron Rovinsky, Vikash Kumar, Abhishek Gupta, and Sergey Levine. Dexterous manipulation from images: Autonomous real-world rl via substep guidance. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. 3

[29] Aditya Kannan, Kenneth Shaw, Shikhar Bahl, Pragna Mannam, and Deepak Pathak. Deft: Dexterous fine-tuning for real-world hand policies. In *Conference on Robot Learning*, 2023. 3

[30] Zheyuan Hu, Aaron Rovinsky, Jianlan Luo, Vikash Kumar, Abhishek Gupta, and Sergey Levine. Reboot: Reuse data for bootstrapping efficient real-world dexterous manipulation. In *7th Annual Conference on Robot Learning*, 2023. 3

[31] Aayush Prakash, Shoubhik Debnath, Jean-Francois Lafleche, Eric Cameracci, Stan Birchfield, Marc T Law, et al. Self-supervised real-to-sim scene generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 16044–16054, 2021. 3

[32] Amlan Kar, Aayush Prakash, Ming-Yu Liu, Eric Cameracci, Justin Yuan, Matt Rusiniak, David Acuna, Antonio Torralba, and Sanja Fidler. Meta-sim: Learning to generate synthetic datasets. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4551–4560, 2019. 3

[33] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv*, 2022. 3

[34] Minghao Guo, Bohan Wang, Pingchuan Ma, Tianyuan Zhang, Crystal Elaine Owens, Chuang Gan, Joshua B Tenenbaum, Kaiming He, and Wojciech Matusik. Physically compatible 3d object modeling from a single image. *arXiv preprint arXiv:2405.20510*, 2024. 3

[35] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Cliport: What and where pathways for robotic manipulation. In *Conference on robot learning*, pages 894–906. PMLR, 2022. 3

[36] Andy Zeng, Pete Florence, Jonathan Tompson, Stefan Welker, Jonathan Chien, Maria Attarian, Travis Armstrong, Ivan Krasin, Dan Duong, Vikas Sindhwani, et al. Transporter networks: Rearranging the visual world for robotic manipulation. In *Conference on Robot Learning*, pages 726–747. PMLR, 2021. 3

[37] Abby O'Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, et al. Open x-embodiment: Robotic learning datasets and rt-x models. *arXiv preprint arXiv:2310.08864*, 2023. 3

[38] Andrew Szot, Bogdan Mazoure, Harsh Agrawal, Devon Hjelm, Zsolt Kira, and Alexander Toshev. Grounding multimodal large language models in actions. *arXiv preprint arXiv:2406.07904*, 2024. 3

[39] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning, 2016. 4, 6

[40] Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. 4

[41] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4015–4026, 2023. 5

[42] Jiaxiang Tang. Stable-dreamfusion: Text-to-3d with stable-diffusion, 2022. https://github.com/ashawkey/stable-dreamfusion. 5

[43] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.*, 41(4):102:1–102:15, July 2022. 5

[44] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2021. 5

[45] Ruoxi Shi, Hansheng Chen, Zhuoyang Zhang, Minghua Liu, Chao Xu, Xinyue Wei, Linghao Chen, Chong Zeng, and Hao Su. Zero123++: a single image to consistent multi-view diffusion base model, 2023. 5

[46] Jacob Munkberg, Jon Hasselgren, Tianchang Shen, Jun Gao, Wenzheng Chen, Alex Evans, Thomas Müller, and Sanja Fidler. Extracting Triangular 3D Models, Materials, and Lighting From Images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8280–8290, June 2022. 5

[47] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects, 2022. 5

[48] Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X. Chang, Leonidas J. Guibas, and Hao Su. SAPIEN: A simulated part-based interactive environment. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020. 5, 7

[49] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning, 2011. 5

[50] Matthias Plappert, Marcin Andrychowicz, Alex Ray, Bob McGrew, Bowen Baker, Glenn Powell, Jonas Schneider, Josh Tobin, Maciek Chociej, Peter Welinder, Vikash Kumar, and Wojciech Zaremba. Multi-goal reinforcement learning: Challenging robotics environments and request for research, 2018. 6

[51] Jonathan D Gammell, Timothy D Barfoot, and Siddhartha S Srinivasa. Batch informed trees (bit*): Informed asymptotically optimal anytime search. *The International Journal of Robotics Research*, 39(5):543–567, 2020. 6

[52] Ioan A. Şucan, Mark Moll, and Lydia E. Kavraki. The Open Motion Planning Library. *IEEE Robotics & Automation Magazine*, 19(4):72–82, December 2012. https://ompl.kavrakilab.org. 6

[53] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018. 6